

Documentação do Sistema – Minisitio v2

> Guia comercial digital – Plataforma de classificados empresariais por cidade/UF

> Versão: 2.1.29 | Stack: Node.js + React + MySQL

Sumário

1. [Visão Geral] (#1-visão-geral)
2. [Arquitetura] (#2-arquitetura)
3. [Stack Tecnológica] (#3-stack-tecnológica)
4. [Estrutura do Projeto] (#4-estrutura-do-projeto)
5. [Banco de Dados] (#5-banco-de-dados)
6. [Backend] (#6-backend)
7. [Frontend] (#7-frontend)
8. [Fluxos de Negócio] (#8-fluxos-de-negócio)
9. [Regras de Negócio] (#9-regras-de-negócio)
10. [API – Endpoints] (#10-api--endpoints)
11. [Deploy e Infraestrutura] (#11-deploy-e-infraestrutura)
12. [Segurança] (#12-segurança)
13. [Crons e Tarefas Agendadas] (#13-crons-e-tarefas-agendadas)
14. [Migração para v3 – Observações] (#14-migração-para-v3--observações)

1. Visão Geral

O **Minisitio** é uma plataforma de guia comercial digital que organiza anúncios empresariais por **cidade (caderno)** e **estado (UF)**. Cada cidade possui um "caderno" onde empresas podem anunciar em diferentes categorias de atividade.

Conceitos-chave

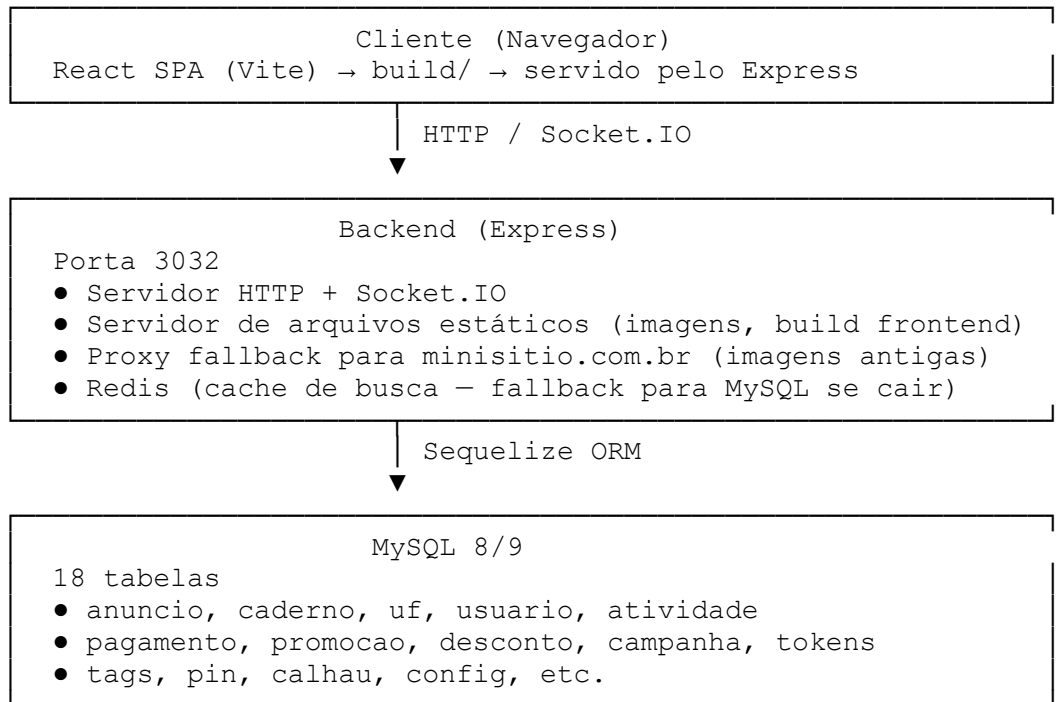
Termo	Definição
--- ---	
Caderno	Diretório de uma cidade. Ex: "ÁGUAS CLARAS - DF"
Anúncio	Ficha de uma empresa dentro de um caderno
Capa	Seção especial de destaque na página inicial do caderno (8 categorias fixas)
Atividade	Categoria do anúncio (ex: "ADMINISTRAÇÃO REGIONAL / PREFEITURA")
Tipo de Anúncio	1=Básico (grátis), 3=Completo (pago), 4=Capa (destaque)
Desconto	Plano/convênio que permite criar anúncios do tipo Capa
Calhau	Frases aleatórias de negócios usadas para balancear colunas no layout
PIN	Código exibido no header do site (portal PIN)

Públicos-alvo

- **Usuários visitantes**: Navegam pelos cadernos, buscam empresas, veem perfis
- **Anunciantes**: Criam e gerenciam seus anúncios (painel do anunciante)
- **Administradores**: Gerenciam todo o sistema via painel admin

2. Arquitetura

...



...

Fluxo de requisição

...

Navegador → Cloudflare (Railway) → Express → [Helmet, CORS, RateLimit]
→ cookie-parser → auth middleware → Routes → Controllers
→ Sequelize → MySQL → Response JSON
→ Cache Redis (opcional, rota de busca)

...

Proxy de Imagens

O backend faz proxy de 4 pastas de imagens para o servidor antigo (`minisitio.com.br`):

Pasta Local	Rota	Servidor Antigo
`public/upload/img/descImagem/`	`/api/files/descImagem/*`	`minisitio.com.br/api/files/descImagem/*`
`public/upload/img/logoParceiro/`	`/api/files/logoParceiro/*`	`minisitio.com.br/api/files/logoParceiro/*`
`public/upload/img/mosaico/`	`/api/files/mosaico/*`	`minisitio.com.br/api/files/mosaico/*`
`public/upload/img/promocao/`	`/api/files/promocao/*`	`minisitio.com.br/api/files/promocao/*`

****Regra**:** Tenta local primeiro. Se não existir, busca no servidor antigo e salva localmente (write-through cache).

3. Stack Tecnológica

Backend (`back/package.json` v10.0.14)

Categoria	Tecnologia
Runtime	Node.js 20 (Alpine Linux em produção)
Framework	Express 4.18
ORM	Sequelize 6.37 (MySQL2)
Autenticação	JWT (jsonwebtoken) + bcryptjs
Cache	Redis 6 (com fallback automático)
Email	Nodemailer (Gmail SMTP)
Pagamento	Mercado Pago SDK 3.1
Upload	Multer 1.4
CSV/XLSX	csv-parser, excel4node, exceljs, @json2csv
PDF	html-pdf, jsPDF (frontend)
Socket.IO	Tempo real (4.8)
Segurança	Helmet, express-rate-limit
Templates	EJS (páginas de email)
Jobs	node-cron

Frontend (`front/package.json` v0.1.0)

Categoria	Tecnologia
Framework	React 18
Bundler	Vite 8
Roteamento	React Router 6
Estado	Context API (3 contextos)
Server State	TanStack React Query 5
CSS	Bootstrap 5, Tailwind 3, CSS Modules
UI Library	Radix UI (dialog, dropdown, tooltip, slot)
Animações	Framer Motion 12
Mapas	@react-google-maps/api
Ícones	Bootstrap Icons, Font Awesome, Lucide React
QR Code	qrcode.react
PDF (front)	jsPDF + html2canvas
PIX	Componente próprio de QR Code
Compartilhar	Capacitor (Web Share API)
Formulários	react-hook-form
Alertas	SweetAlert2
Cookies	js-cookie

Infraestrutura

Componente	Tecnologia
Container	Docker (multi-stage)
Orquestração	Docker Compose (dev)
Produção	Railway.app
Proxy	Cloudflare (Railway Hikari)
Banco	MySQL 8/9 (Railway volume)
Cache	Redis (Railway, opcional)

4. Estrutura do Projeto

...

```
minisitio/
├── back/                                     # Backend Express
│   ├── index.js                             # Entry point (399 linhas)
│   ├── package.json
│   ├── .env.example
│   ├── Dockerfile
│   ├── config/
│   │   ├── db.js                           # Sequelize MySQL config
│   │   ├── redis.js                        # Redis client (com fallback)
│   │   ├── config.js                       # API keys, Mercado Pago
│   │   └── smtp.js                         # Nodemailer config
│   ├── models/                             # 19 modelos Sequelize
│   │   ├── table_anuncio.js               # Anúncios (657 linhas)
│   │   ├── table_caderno.js               # Cidades/cadernos
│   │   ├── table_uf.js                    # Estados
│   │   ├── table_usuarios.js              # Usuários
│   │   ├── table_atividade.js             # Categorias
│   │   ├── table_pagamentos.js            # Pagamentos
│   │   ├── table_desconto.js              # Planos/convênios
│   │   ├── table_promocao.js              # Promoções/banners
│   │   ├── tokens_promocao.js             # Tokens de campanha
│   │   ├── table_tags.js                  # Tags
│   │   ├── table_pin.js                   # PIN codes
│   │   ├── table_inst_config.js           # Config institucional
│   │   ├── table_contato_config.js        # Config contato
│   │   ├── table_campanha.js              # Campanhas
│   │   ├── table_calhau.js                # Frases de negócios
│   │   ├── table_importStage.js           # Stage de importação
│   │   ├── table_globals.js               # Chave-valor global
│   │   ├── table_ddd.js                   # DDDs brasileiros
│   │   └── table_user_login.js            # Login (versão simplificada)
│   ├── controllers/
│   │   ├── Admin.js                       # Admin CRUD principal (58KB)
│   │   ├── Login.js                       # Autenticação
│   │   ├── Users.js                       # CRUD usuários
│   │   ├── Buscador.js                    # Busca com Redis
│   │   ├── WebHooks.js                    # Webhook Mercado Pago
│   │   ├── RecuperarSenha.js              # Reset de senha
│   │   ├── BemVindo.js                    # Landing page data
│   │   ├── Rotinas.js                     # Rotinas administrativas
│   │   ├── Upload.js                      # Upload de arquivos
│   │   ├── UserActions.js                 # Ações do usuário
│   │   ├── Email.js                       # Disparo de emails
│   │   ├── identificarNuDoc.js             # Identificador CPF/CNPJ
│   │   └── ImportController.js             # Importação CSV
│   └── admin/
│       ├── espacos/index.js               # Controller principal (7350 linhas)
│       ├── campanha/index.js              # Geração de campanhas
│       ├── institucional-config/controller.js
│       └── contato-config/controller.js
├── routes/
│   └── Routes.js                           # 70+ rotas
├── middlewares/
│   ├── authentication.js                   # JWT decode → req.user
│   └── authVerification.js                 # JWT decode → req.userId
```

└─	uploadImage.js	# Multer (imagens)
└─	uploadPdf.js	# Multer (PDFs)
└─	crons/	
└─	└─ promocao.js	# Expurgo de promoções
└─	└─ campanha.js	# Expurgo de campanhas
└─	public/	
└─	└─ upload/	
└─	└─ └─ img/	
└─	└─ └─ └─ descImagem/	# Imagens dos anúncios
└─	└─ └─ └─ mosaico/	# Imagens de fundo dos cadernos
└─	└─ └─ └─ logoParceiro/	# Logos de parceiros
└─	└─ └─ └─ logoCertificado/	
└─	└─ └─ └─ imgCertificado/	
└─	└─ └─ └─ logoCashBack/	
└─	└─ └─ └─ promocao/	# Banners de promoção
└─	└─ └─ └─ adminInstitucional/	
└─	sendMailer.js	# 6 funções de email
└─	server.js	# Export Excel
└─	serverExportUser.js	
└─	serverExportId.js	
└─	exportExcellCaderno.js	
└─	exportExcellAtividade.js	
└─	serverImport.js	# Import CSV
└─	archiver.js	# ZIP builder
└─	moveAndRename.js	# Renomear arquivos
└─	validations.js	# CPF, CNPJ, email validators
└─	backup-database.js	# Backup MySQL
└─	run-queries.js	# Diagnostic queries
└─	seed-user.js	# Seeder admin
└─	ejs/	# Templates de email
└─	migrations/	# Scripts de migração
└─	front/	# Frontend React
└─	└─ vite.config.js	# Vite bundler config
└─	└─ package.json	# 57+ dependências
└─	└─ index.html	# Entry HTML
└─	└─ tailwind.config.js	
└─	└─ postcss.config.js	
└─	└─ src/	
└─	└─ └─ index.jsx	# Mount + providers
└─	└─ └─ App.jsx	# Root layout
└─	└─ └─ routes/	
└─	└─ └─ └─ Rotas.jsx	# 50+ rotas
└─	└─ └─ └─ PrivateRoute.jsx	# Guard de autenticação
└─	└─ └─ config/	
└─	└─ └─ └─ config.jsx	# API URLs (auto-detect)
└─	└─ └─ └─ capaFallbacks.jsx	# Fallback de capas por UF
└─	└─ └─ context/	
└─	└─ └─ └─ AuthContext.jsx	# Estado de autenticação
└─	└─ └─ └─ BuscaContext.jsx	# Estado de busca
└─	└─ └─ └─ QrcodeCadernoContext.jsx	# Tema QR code
└─	└─ └─ views/	# 20+ views
└─	└─ └─ └─ Home.jsx	
└─	└─ └─ └─ Caderno.jsx	# (1504 linhas)
└─	└─ └─ └─ CadernoGeral.jsx	# Página de capa
└─	└─ └─ └─ WebCard.jsx	# Perfil da empresa
└─	└─ └─ └─ Pesquisa.jsx	# Resultados de busca
└─	└─ └─ └─ ComprarAnuncio.jsx	# Compra de anúncio (1107 linhas)

- Login.jsx
- NotFound.jsx
- classificados/
 - CardClassificado.jsx
 - Letter.jsx
- cadernos/geral/Caderno.jsx
- painelAnunciante/ # Painel do anunciante
- comprar-anuncio/ # Formulários de compra
- infoPages/ # Institucional, Contato, Privacidade
- promocao/Pesquisa.jsx
- campanha/Promocao.jsx
- area-assinante/AssinanteCadastro.jsx
- components/ # 39+ componentes
 - Busca.jsx # (575 linhas)
 - Mosaico.jsx # Header principal
 - FullWebCard.jsx # Card de perfil completo
 - MiniWebCard.jsx # Card compacto (442 linhas)
 - MiniWebCardSimples.jsx
 - Cardlist.jsx # Item de lista de busca
 - ButtonCapa.jsx
 - SafeMosaico.jsx # Imagem com fallback
 - Nav.jsx / Footer.jsx
 - Resultados.jsx
 - Navegacao.jsx # Breadcrumb
 - Metadados.jsx # Metadados do perfil (294 linhas)
 - ContactForm.jsx # Formulário de contato (193 linhas)
 - MapContainer.jsx # Google Maps Street View
 - Loading.jsx
 - UserActions.jsx # Ações do perfil (348 linhas)
 - MosaicoWebCard.jsx
 - WebcardThumb.jsx
 - Tooltip.jsx
 - Pagination.jsx
 - MsgProgramada.jsx
 - Modal/ # Modais de login/cadastro
 - promocao/ # Componentes de promoção
 - localidade/ # Seletores de localidade
- admin/ # 25+ arquivos do painel admin
 - Administrator.jsx
 - view/
 - Dashboard.jsx
 - usuarios/
 - cadernos/
 - Espacos/
 - Atividades/
 - Pagamentos/
 - Pin/
 - Calhau/
 - Campanha/
 - ConfiguracoesPortal/
 - Duplicidades/
 - BuscarProfissionais/
 - components/ # Componentes admin reutilizáveis
- plugins/ # 6 plugins
- layouts/Layout.jsx # Layout admin com sidebar
- api/admin/espacos.jsx # Helpers de API
- globalFunctions/functions.jsx
- hooks/ # Custom hooks

```

├── ┌── assets/           # CSS, imagens
│   ├── public/         # Static assets
│   └── build/           # Output de build (produção)
├── .railway/
│   └── railway.ts       # Config Railway IaC
├── Dockerfile           # Docker multi-stage (root)
├── docker-compose.yml   # Dev orchestration
├── railway.json         # Config Railway (builder=DOCKERFILE)
├── .dockerignore
└── DOCUMENTACAO_SISTEMA.md # Este arquivo
...

```

5. Banco de Dados

5.1 Diagrama de Relacionamentos

...

```

uf (id_uf)
├── caderno (codUf)
├── anuncio (codUf)           [valor real: sigla 'DF' ou id 53 -
inconsistente]
├── usuario (codUf)           [STRING, não é FK formal]
└── ddd (id_uf)

caderno (codCaderno)
├── anuncio (codCaderno)     [valor real: nome da cidade, não o ID]
└── importStage (codCaderno)

usuario (codUsuario)
├── desconto (idUsuario)
└── anuncio (descCPF/CNPJ)   [FK não-constraint via CPF/CNPJ]

atividade (id)
├── anuncio (codAtividade)
└── anuncio → atividade.nomeAmigavel (alias: atividadeAmigavel)

desconto (idDesconto)
├── campanhas (idPromocional)
└── campanhas (idRetorno)

desconto (hash)
├── anuncio (codDesconto)
└── campanhas (via desconto)

anuncio (codAnuncio)
├── pagamento (ref_mp_codAnuncio)
├── promocao (codAnuncio)
├── tokens_promocao (codAnuncio)
└── tags (codAnuncio)        [STRING, não é FK formal]
...

```

5.2 Lista de Tabelas (18)

#	Tabela	Finalidade	Linhas (aprox)
---	---	---	---

1	`anuncio`	Anúncios de empresas	~200K	
2	`caderno`	Cidades brasileiras (directory)	5.628	
3	`uf`	Estados brasileiros (27)	27	
4	`usuario`	Usuários da plataforma	~50K	
5	`atividade`	Categorias de atividade	~50	
6	`pagamento`	Transações Mercado Pago	Variável	
7	`promocao`	Banners promocionais	Variável	
8	`desconto`	Planos/convênios de desconto	Variável	
9	`importStage`	Stage de importação (clone do anuncio)	Temporário	
10	`tokens_promocao`	Tokens de campanha promocional	Variável	
11	`tags`	Tags de anúncios	Variável	
12	`pin`	Códigos PIN do portal	Variável	
13	`institucional_config`	Config da página institucional	1	
14	`contato_config`	Config da página de contato	1	
15	`campanhas`	Campanhas de marketing	Variável	
16	`calhau`	Frases de negócios para layout	Variável	
17	`globals`	Chave-valor global	Variável	
18	`ddd`	DDDs brasileiros	~200	

5.3 Tabelas Principais (detalhado)

`anuncio` – Anúncios

Colunas críticas: `codAnuncio` (PK), `codUsuario`, `codTipoAnuncio` (1=básico, 3=completo, 4=capa), `codAtividade`, `codCaderno` (armazena NOME da cidade, string), `codUf` (armazena SIGLA, string), `descImagem` (nome do arquivo de imagem), `activate` (0/1), `codDesconto`, `dueDate` (criação + 365 dias), `page`, `codDuplicado`.

****Índices**:** `codUf`, `codCaderno`, `activate`, `createdAt`, composto `(codUf, codCaderno, activate)`, composto `(codCaderno, codAtividade)`, `codUsuario`, `codDuplicado`.

****Hook**:** `beforeCreate` seta `dueDate = createdAt + 365 days`.

`caderno` – Cidades/Cadernos

Colunas: `codCaderno` (PK), `codUf` (FK → uf), `UF` (sigla), `nomeCaderno` (nome completo maiúsculo), `nomeCadernoFriendly` (URL-friendly), `descImagem` (arquivo de imagem do mosaico), `cep\_inicial`/`cep\_final`, `isCapital` (0=capital, 1=interior), `legenda`, `basico` (contador), `completo` (contador), `total` (contador geral).

****Regra**:** `isCapital = 0` significa que é capital. Capitais têm `descImagem` preenchida.

`usuario` – Usuários

Colunas: `codUsuario` (PK), `descCPF` (unique), `senha` (bcrypt), `codTipoUsuario` (1=admin, 2=anunciante, 3=anunciante restrito), `resetToken`/`resetTokenExpires`, `ativo` (0/1).

6. Backend

6.1 Entry Point (`back/index.js`)

Ordem de inicialização:

1. `dotenv.config()`
2. Handlers globais: `unhandledRejection` (log), `uncaughtException` (log + `process.exit(1)`)
3. Validação de variáveis de ambiente obrigatórias (`API\_KEY`, `API\_SECRET`, `DB\_PASSWORD` ou `DATABASE\_URL`)
4. Criação do servidor HTTP + Socket.IO
5. Configuração CORS (dynamic origins + Railway domain)
6. Helmet (CSP configurado para scripts e imagens)
7. Rate limiter global (100 req/min)
8. cookie-parser
9. EJS (views de email)
10. Static mounts (imagens, frontend build)
11. Proxy fallback de imagens (4 pastas)
12. Rotas
13. Login rate limiter (10 tentativas / 15 min)
14. Socket.IO configuração
15. Crons (diários: 03:00, 04:00)
16. Error handler 404 → SPA fallback (`index.html`)
17. Error handler 500 genérico

6.2 Controllers Principais

`controllers/admin/espacos/index.js` (7350 linhas)

O maior arquivo do sistema. Contém **35+** funções para CRUD de anúncios:

Função	Descrição
`cadastrear(req, res)`	Cria anúncio com validações
`listarClassificado(req, res)`	Retorna capas + listagem paginada de um caderno
`listarClassificadoGeral(req, res)`	Listagem paginada sem capas
`listarTodosClassificados(req, res)`	Todos os classificados sem paginação
`read(req, res)`	Leitura com filtros (UF, caderno, atividade, página)
`readOne(req, res)`	Anúncio individual por ID
`editar(req, res)`	Edição de anúncio
`deletar(req, res)`	Exclusão
`importCSV(req, res)`	Importação via CSV
`exportXLSX(req, res)`	Exportação via XLSX
`visuData(req, res)`	Dados de visualizações
`quantidadeUf(req, res)`	Quantidade por UF (dashboard)
`reativar(req, res)`	Reativação de anúncio
`mudarPagina(req, res)`	Mudar página de um anúncio
`duplicarAnuncio(req, res)`	Duplicar anúncio
`unificarAnuncio(req, res)`	Unificar duplicados
`listaTest(req, res)`	Lista para teste
+ funções de stage import, contagem, etc.	

`controllers/Admin.js` (58KB)

43 funções exportadas para CRUD de:

- Cadernos, Atividades, Usuários, Descontos, Promoções
- Globals, DDDs, UF, Tags, Tokens

- Pagamentos, Pins, Calhaus
- Operações bulk, importação CSV, exportação Excel
- Gerenciamento de campanhas

`controllers/Buscador.js`

Busca textual com múltiplos parâmetros:

- `term` (texto de busca)
- `uf`, `caderno`, `atividade`
- `page`, `limit`
- Cache Redis com TTL de 1 hora
- Fallback automático para MySQL se Redis cair

`controllers/Login.js`

- Validação de CPF/CNPJ
- bcrypt compare
- Geração de JWT (expiração configurável)
- Rate limit (10 tentativas / 15 min)
- Cookie HTTPOnly + JWT response
- Registro de tentativas

`controllers/WebHooks.js`

- Webhook Mercado Pago
- Verificação de assinatura HMAC
- Processamento de pagamento (status `approved`)
- Ativação de token de campanha
- Email de confirmação no primeiro pagamento aprovado

`controllers/RecuperarSenha.js`

- Geração de token (expira 1 hora)
- Envio de email com link de reset
- Validação do token
- Atualização da senha (bcrypt)

6.3 Middlewares

Middleware	Função
`authentication.js`	Decodifica JWT → `req.user = { id, nome }`
`authVerification.js`	Decodifica JWT → `req.userId` (uuid)
`uploadImage.js`	Multer: 5MB, PNG/JPG/JPEG, pastas: `logoParceiro`, `logoCertificado`, `imgCertificado`, `logoCashBack`, `descImagem`, `promocao`
`uploadPdf.js`	Multer: 5MB, PDF, pasta: `cartaoDigital`

6.4 Funções de Email (`sendMailer.js`)

Função	Disparo
`faleComDono(dados)`	Contato do perfil → dono do anúncio
`faleComDonoCliente(dados)`	Cópia para o cliente
`contato(dados)`	Formulário de contato do site
`novoUsuario(dados)`	Novo cadastro de usuário
`forgotPasswordEmail(dados)`	Email de recuperação de senha
`enviarPromocao(dados)`	Email promocional

7. Frontend

7.1 Providers (ordem de aninhamento)

...

```
React.StrictMode
  QueryClientProvider (TanStack)
    AuthProvider
      BrowserRouter
        App
          TemaProvider (BuscaContext)
            Routes
              QrcodeCadernoProvider (somente Caderno e CadernoGeral)
```

...

7.2 Rotas Públicas

Rota	Componente	Função
('/')	Home	Landing page com busca
('/buscar/:caderno/:estado')	Pesquisa	Resultados da busca
('/caderno/:atividade')	Caderno	Caderno classificado (2 colunas)
('/cadernos/:atividade')	TodosCaderno	Todos os cadernos
('/caderno-geral/:caderno/:estado')	CadernoGeral	Capa do caderno (8 categorias)
('/perfil/:codAnuncio')	WebCard	Perfil completo da empresa
('/login')	Login	Autenticação
('/promocoes/:caderno/:estado')	Promocoes	Busca de promoções
('/promocao/:hash')	Promocao	Landing page de campanha
('/comprar-espaco-minisitio')	ComprarAnuncio	Compra de anúncio
('/anuncie')	ComprarAnuncio	Alias
('/criar-cadastro')	AssinanteCadastro	Cadastro
('/renovar/perfil/:codAnuncio')	AtualizarPerfil	Renovação
('/forgot-password')	ForgotPassword	Recuperar senha
('/reset-password')	ResetPassword	Resetar senha
('/qrcode')	Qrcode	Plugin QR Code
('/adesivo')	Adesivo	Plugin adesivo
('/institucional')	Institucional	Sobre
('/contato')	Contato	Fale conosco
('/politica-privacidade')	PoliticaPrivacidade	Privacidade
('*')	NotFound	404

7.3 Rotas Admin (protegidas por `PrivateRoute`)

Todas sob `/admin/\*` com layout de sidebar (`Layout.jsx`):

- `/admin/dashboard`
- `/admin/users`, `/admin/usuarios/cadastro`, `/admin/usuarios/editar`
- `/admin/Cadernos`, `/admin/cadernos/cadastro`,
`/admin/cadernos/editar`, `/admin/info/Cadernos`
- `/admin/atividades`, `/admin/atividades/cadastro`,
`/admin/atividades/editar`
- `/admin/desconto`, `/admin/desconto/cadastro`, `/admin/desconto/editar`
- `/admin/espacos`, `/admin/anuncio/cadastro`, `/admin/anuncio/editar`,
`/admin/anuncio/import`
- `/admin/pagamentos`, `/admin/pagamento/config`

- `/admin/pin`, `/admin/pin/cadastro`, `/admin/pin/editar`
- `/admin/buscar-profissionais`
- `/admin/calhau`, `/admin/calhau/cadastro`
- `/admin/configuracoes`, `/admin/institucional`, `/admin/contato`
- `/admin/duplicidades`
- `/admin/campanha`

7.4 Contextos

Contexto	Estado	Ações	Persistência
`AuthContext`	`{ user, loading }`	`login()`, `logout()`	JWT em sessionStorage
`BuscaContext`	`{ result }`	`setResult()`	Memória (volátil)
`QrcodeCadernoContext`	`{ theme }`	`toggleTheme()`	Memória

7.5 Componentes Críticos

Componente	Linhas	Função
`Busca.jsx`	575	Busca principal com seleção UF/cidade, autocomplete, geolocalização
`Caderno.jsx`	1504	Layout 2 colunas com balanceamento, paginação, scroll-to-item
`CadernoGeral.jsx`	341	Capa do caderno com 8 cards de categoria
`ComprarAnuncio.jsx`	1107	Formulário de compra com preview, desconto, 3 tipos de anúncio
`FullWebCard.jsx`	379	Perfil completo da empresa
`MiniWebCard.jsx`	442	Card compacto (listagem)
`Metadados.jsx`	294	Metadados + redes sociais + PIX
`UserActions.jsx`	348	Ações: renovar, compartilhar, QR code, denúncia
`ContactForm.jsx`	193	Formulário "Fale com o dono"

8. Fluxos de Negócio

8.1 Navegação do Visitante

...

Home → Busca (UF + Cidade) → CadernoGeral (capa)
 → Ver caderno classificado → Caderno (lista 2 colunas)
 → Clicar em anúncio → WebCard (perfil completo)
 → Ações: contato, mapa, site, redes sociais, compartilhar

...

8.2 Fluxo de Capa (CadernoGeral)

...

Requisição: GET /api/admin/anuncio/classificado/:caderno/:estado

1. Decodifica parâmetros
2. Busca caderno na tabela `caderno` (por nome ou friendly name + UF)
3. Se não encontrar por nome, tenta por codCaderno (inteiro)
4. Se não encontrar → retorna vazio
5. Busca anúncios de capa (8 tipos fixos de atividade) → `capas`
6. Busca todos anúncios (paginado, 10 itens) → `teste`

7. Retorna: { capas, teste, mosaico (descImagem), totalRegistros }

Frontend:

- Exibe mosaico (imagem de fundo do caderno)
 - Renderiza 8 cards (CardClassificado) com os dados de `capas`
 - Se `capas[i]` existe, mostra a imagem do anúncio
 - Se a imagem falha (onError), usa fallback da UF (capaFallbacks.js)
 - Se não existe anúncio, mostra "NÃO INFORMADO"
- ...

8.3 Fluxo de Compra de Anúncio

...

ComprarAnuncio.jsx:

1. Usuário preenche formulário:
 - Tipo: Básico (grátis), Completo (pago), Capa (convênio)
 - Dados da empresa, endereço, contato
 - Categoria de atividade
 - Desconto (opcional, formato 99.999.9999)
 2. Validações:
 - CPF/CNPJ uniqueness
 - Desconto válido (via API)
 - Campos obrigatórios por tipo
 3. Se for pago:
 - Redireciona para Mercado Pago
 - Webhook processa pagamento
 4. Se aprovado:
 - Anúncio criado com `activate = 1`
 - Due date = data atual + 365 dias
- ...

8.4 Fluxo de Pagamento (Mercado Pago)

...

1. Usuário clica em "Comprar" → POST /pagamento/create
 2. Backend gera URL de pagamento MP
 3. Usuário paga no ambiente MP
 4. MP envia webhook → POST /api/webhook/mercadopago
 5. Backend verifica assinatura, atualiza status
 6. Se approved:
 - Atualiza `pagamento.status`
 - Se campanha: ativa token de promoção
 - Dispara email de confirmação
- ...

8.5 Fluxo de Busca

...

Busca.jsx:

1. Usuário seleciona UF → GET /ufs
2. Seleciona cidade → GET /cadernos?uf=
3. Opcional: digita termo de busca
4. Clica buscar → POST /api/buscar
Body: { term, uf, caderno, atividade, page, limit }
5. Backend Buscador.js:
 - Tenta Redis cache (chave = hash dos parâmetros)
 - Se cache miss: consulta MySQL com filtros

- JOIN com atividade (nomeAmigavel)
- Cacheia resultado por 1 hora
- Retorna paginado

6. Frontend: renderiza Cardlist para cada resultado
```

### ### 8.6 Fluxo de Autenticação

```

Login.jsx:

1. Usuário insere CPF/CNPJ + senha
2. POST /api/login
3. Backend:
 - Valida CPF/CNPJ
 - Busca usuário por descCPFCNPJ
 - bcrypt.compare(senha)
 - Gera JWT com { id, nome, codTipoUsuario }
 - Seta cookie httpOnly + sameSite strict
 - Retorna { success, token, user }
4. Frontend:
 - Salva token em sessionStorage
 - AuthContext.setUser(user)
 - Redireciona conforme tipo de usuário

```

---

## ## 9. Regras de Negócio

### ### 9.1 Tipos de Anúncio

| Código | Nome     | Preço    | Recursos                                                 |
|--------|----------|----------|----------------------------------------------------------|
| 1      | Básico   | Grátis   | Texto + telefone. Sem imagem, mapa, redes sociais        |
| 3      | Completo | Pago     | Todos recursos: imagem, mapa, redes, PIX, certificado    |
| 4      | Capa     | Convênio | Destaque na capa do caderno. Requer `desconto` tipo capa |

### ### 9.2 Regras de Capa

- Existem exatamente **8** categorias de capa:
  1. ADMINISTRAÇÃO REGIONAL / PREFEITURA
  2. EMERGÊNCIA
  3. UTILIDADE PÚBLICA
  4. HOSPITAIS PÚBLICOS
  5. CÂMARA DE VEREADORES - CÂMARA DISTRITAL
  6. SECRETARIA DE TURISMO
  7. INFORMAÇÕES
  8. EVENTOS NA CIDADE
- Anúncios de capa são do tipo `codTipoAnuncio = 4`
- Capas só podem ser criadas com desconto onde `desconto.is\_capa = true`
- A página de capa mostra até 1 anúncio por categoria (ordenado por nome)
- Anúncios de capa NÃO aparecem na listagem regular do caderno

### ### 9.3 Regras de Exibição no Caderno

- Layout de **\*\*2 colunas\*\*** com balanceamento de altura
- Anúncios são distribuídos entre as colunas `base1` e `base2`
- Sistema de **\*\*"calhau"\*\*: frases aleatórias preenchem espaço para** igualar alturas
- Paginação: 10 itens por página
- Scroll-to-item: se URL tem `?id=...`, destaca o item com borda pulsante
- Anúncios de capa são filtrados da listagem regular

### ### 9.4 Regras de Usuário

- `codTipoUsuario = 1`: Super admin (acesso total)
- `codTipoUsuario = 2`: Anunciante (painel do anunciante)
- `codTipoUsuario = 3`: Anunciante restrito (bloqueado em alguns CRUDs admin)
- CPF/CNPJ é único
- Senha armazenada com bcrypt
- Reset de senha: token com expiração de 1 hora

### ### 9.5 Regras de Imagem

- Upload: 5MB máximo, PNG/JPG/JPEG
- Imagens de descImagem e logos em pastas separadas
- Se a imagem não existe localmente, busca no servidor antigo (proxy fallback)
- Se o proxy falha, frontend usa fallback (placeholder ou foto da UF)

### ### 9.6 Regras de Cache (Redis)

- Busca textual: cache por 1 hora
- Chave = hash de todos os parâmetros de busca
- Se Redis cai, fallback automático para MySQL
- Apenas a rota de busca usa cache

### ### 9.7 Regras de Promoção

- Promoções expiram em `data\_validade`
- Cron diário (03:00) deleta promoções expiradas
- Campanhas expiram em `dataFim`
- Cron diário (04:00) inativa campanhas expiradas e rebaixa tokens para "vencido"

### ### 9.8 Regras de Due Date

- Anúncios têm `dueDate = createdAt + 365 dias`
- Definido em hook `beforeCreate` do modelo
- Campos `basico`, `completo`, `total` na tabela `caderno` são contadores incrementados manualmente

---

## ## 10. API — Endpoints

### ### 10.1 Autenticação

| Método | Rota           | Controller  | Descrição        |
|--------|----------------|-------------|------------------|
| POST   | `/api/login`   | Login.login | Autenticação     |
| GET    | `/api/is-auth` | -           | Valida token JWT |

### ### 10.2 Anúncios (Espaços)

| Método | Rota                                                 | Descrição                     |
|--------|------------------------------------------------------|-------------------------------|
| GET    | `/api/admin/anuncio/classificado/:caderno/:uf`       | Capas + listagem paginada     |
| GET    | `/api/admin/anuncio/classificado/geral/:caderno/:uf` | Listagem paginada (sem capas) |
| GET    | `/api/admin/anuncio/classificado/todos/:caderno/:uf` | Todos anúncios                |
| GET    | `/api/admin/espacos/read`                            | Anúncios com filtros          |
| GET    | `/api/anuncio/:id`                                   | Anúncio individual            |
| POST   | `/api/admin/anuncio/create`                          | Criar anúncio                 |
| PUT    | `/api/admin/anuncio/update/:id`                      | Atualizar anúncio             |
| DELETE | `/api/admin/anuncio/delete/:id`                      | Deletar anúncio               |
| POST   | `/api/admin/anuncio/import`                          | Importar CSV                  |
| GET    | `/api/admin/anuncio/export`                          | Exportar XLSX                 |
| GET    | `/api/admin/anuncio/quantidade/uf`                   | Quantidade por UF             |

### ### 10.3 Busca

| Método | Rota          | Descrição                       |
|--------|---------------|---------------------------------|
| POST   | `/api/buscar` | Busca textual (com cache Redis) |

### ### 10.4 Cadernos

| Método | Rota                      | Descrição         |
|--------|---------------------------|-------------------|
| GET    | `/api/cadernos`           | Listar cadernos   |
| GET    | `/api/cadernos/:uf`       | Cadernos por UF   |
| POST   | `/api/caderno/create`     | Criar caderno     |
| PUT    | `/api/caderno/update/:id` | Atualizar caderno |
| DELETE | `/api/caderno/delete/:id` | Deletar caderno   |

### ### 10.5 UFs

| Método | Rota                 | Descrição    |
|--------|----------------------|--------------|
| GET    | `/api/ufs`           | Listar UFs   |
| POST   | `/api/uf/create`     | Criar UF     |
| PUT    | `/api/uf/update/:id` | Atualizar UF |

### ### 10.6 Atividades

| Método | Rota                              | Descrição        |
|--------|-----------------------------------|------------------|
| GET    | `/api/atividade/:id`              | Atividade por ID |
| POST   | `/api/admin/atividade/create`     | Criar atividade  |
| PUT    | `/api/admin/atividade/update/:id` | Atualizar        |
| DELETE | `/api/admin/atividade/delete/:id` | Deletar          |

### ### 10.7 Usuários

| Método | Rota                        | Descrição     |
|--------|-----------------------------|---------------|
| POST   | `/api/admin/usuario/create` | Criar usuário |



| Método | Rota                                 | Descrição       |
|--------|--------------------------------------|-----------------|
| PUT    | `/api/admin/usuario/update/:id`      | Atualizar       |
| GET    | `/api/admin/usuario/read`            | Listar usuários |
| GET    | `/api/admin/usuario/read/:id`        | Usuário por ID  |
| POST   | `/api/recuperar-senha`               | Solicitar reset |
| POST   | `/api/validar-token-recuperar-senha` | Validar token   |
| POST   | `/api/atualizar-senha`               | Atualizar senha |

### ### 10.8 Pagamentos

| Método | Rota                        | Descrição          |
|--------|-----------------------------|--------------------|
| POST   | `/api/pagamento/create`     | Criar pagamento MP |
| POST   | `/api/webhook/mercadopago`  | Webhook MP         |
| GET    | `/api/admin/pagamento/read` | Listar pagamentos  |

### ### 10.9 Descontos

| Método | Rota                                 | Descrição        |
|--------|--------------------------------------|------------------|
| GET    | `/api/admin/desconto/read/all`       | Listar todos     |
| GET    | `/api/admin/desconto/aplicar/:codId` | Aplicar desconto |
| POST   | `/api/admin/desconto/create`         | Criar            |
| PUT    | `/api/admin/desconto/update/:id`     | Atualizar        |

### ### 10.10 Arquivos

| Método | Rota                           | Descrição                 |
|--------|--------------------------------|---------------------------|
| POST   | `/api/upload-image`            | Upload imagem (multer)    |
| POST   | `/api/upload-pdf`              | Upload PDF (multer)       |
| GET    | `/api/files/:folder/:filename` | Servir arquivos estáticos |

### ### 10.11 Configurações

| Método   | Rota                              | Descrição            |
|----------|-----------------------------------|----------------------|
| GET/POST | `/api/admin/institucional/config` | Config institucional |
| GET/POST | `/api/admin/contato/config`       | Config contato       |
| GET/POST | `/api/portal/pin`                 | PIN code             |

### ### 10.12 Campanhas

| Método | Rota                        | Descrição                |
|--------|-----------------------------|--------------------------|
| POST   | `/api/admin/campanha/gerar` | Gerar campanha           |
| GET    | `/api/admin/campanha/read`  | Listar campanhas         |
| GET    | `/api/promocao/:hash`       | Landing page de campanha |

---

## ## 11. Deploy e Infraestrutura

### ### 11.1 Docker (Desenvolvimento Local)

```

```bash
# Subir ambiente completo
docker-compose up --build

```

```
# Serviços:
# - backend: porta 3032
# - frontend: porta 3001 (nginx)
# - db: porta 3307 (MySQL 8)
````
```

### ### 11.2 Railway (Produção)

```
Config: `railway.json` + `.railway/railway.ts`

- **Builder**: Dockerfile (multi-stage)
- **Node**: 20-alpine
- **Porta**: 3032
- **Banco**: MySQL 9.4 (volume persistente)
- **Domínio**: `minisitio-v2-production.up.railway.app`
- **Domínio customizado**: `minisitio.com.br` (via Cloudflare)

Volumes: `/app/back/public/upload` (imagens persistentes)

Comandos:
```bash
railway status          # Status do projeto
railway deployment list # Listar deployments
railway deployment redeploy --yes # Redeploy
railway logs            # Logs do serviço
```
```

### ### 11.3 Variáveis de Ambiente

| Variável          | Obrigatória | Descrição                               |
|-------------------|-------------|-----------------------------------------|
| `API_KEY`         | Sim         | Chave da API (mín 10 chars em produção) |
| `API_SECRET`      | Sim         | Segredo JWT (mín 64 chars em produção)  |
| `DATABASE_URL`    | Sim*        | URL MySQL Railway                       |
| `MYSQL_URL`       | Sim*        | Fallback URL MySQL                      |
| `DB_HOST`         | Não         | Host MySQL                              |
| `DB_USER`         | Não         | Usuário MySQL                           |
| `DB_PASSWORD`     | Sim*        | Senha MySQL (*se sem DATABASE_URL)      |
| `DB_NAME`         | Não         | Nome do banco                           |
| `JWT_SECRET`      | Sim         | Segredo JWT                             |
| `JWT_EXPIRES`     | Não         | Expiração JWT (ex: 7d)                  |
| `MP_ACCESS_TOKEN` | Não         | Token Mercado Pago                      |
| `MP_PUBLIC_KEY`   | Não         | Chave pública MP                        |
| `SMTP_HOST`       | Não         | Servidor SMTP                           |
| `SMTP_USER`       | Não         | Usuário SMTP                            |
| `SMTP_PASS`       | Não         | Senha SMTP                              |
| `ALLOWED_ORIGINS` | Não         | Origens CORS adicionais                 |
| `REDIS_URL`       | Não         | URL Redis                               |
| `NODE_ENV`        | Não         | production/development                  |

---

## ## 12. Segurança

### ### 12.1 Medidas Implementadas

| Medida | Localização |
|--------|-------------|
|        | ---         |

```
Helmet (CSP, XSS, etc.)	`back/index.js`
Rate limit global (100 req/min)	`back/index.js`
Rate limit login (10/15min)	`back/index.js`
CORS com whitelist	`back/index.js`
JWT com httpOnly + secure + sameSite	`back/index.js`
Validação de env vars na inicialização	`back/index.js`
Proteção LFI (path.resolve + startsWith)	`back/index.js` (proxy)
bcrypt para senhas	`controllers/Login.js`
Validação de CPF/CNPJ	`validations.js`
Multer (tamanho, tipo de arquivo)	`middlewares/uploadImage.js`
Process.exit(1) em crash	`back/index.js`
Backup assíncrono (não bloqueia event loop)	`backup-database.js`
```

### ### 12.2 CSP (Content Security Policy)

Configurada via Helmet para permitir:

- Google Maps, Google Fonts, Google APIs
- Mercado Pago SDK
- Scripts e estilos do próprio domínio
- Imagens de qualquer origem (necessário para proxy de imagens)

---

## ## 13. Crons e Tarefas Agendadas

### ### 13.1 Promoções Expiradas (`crons/promocao.js`)

- **\*\*Agenda\*\***: Diário às 03:00
- **\*\*Ação\*\***: Deleta registros da tabela `promocao` onde `data\_validade < NOW()`
- **\*\*Hook\*\***: `beforeDestroy` no modelo deleta o arquivo de banner do disco

### ### 13.2 Campanhas Expiradas (`crons/campanha.js`)

**\*\*Três tarefas\*\*** executadas em sequência diariamente às 04:00:

1. **\*\*`inativarCampanhasExpiradas()`\*\***: Atualiza `campanhas.status = 'INATIVO'` onde `dataFim < NOW()`
2. **\*\*`downgradePerfil()`\*\***: Atualiza `tokens\_promocao.statusPromocao = 'vencido'` para tokens cuja `dataLimitePromocao` já passou
3. **\*\*`downgradePerfil()` (segunda parte)\*\***: Rebaixa anúncios de `codTipoAnuncio = 3` (completo) para `codTipoAnuncio = 1` (básico) quando o token associado está vencido

---

## ## 14. Migração para v3 – Observações

### ### 14.1 Pontos de Atenção

1. **\*\*Modelo inconsistente\*\***: `codUf` e `codCaderno` na tabela `anuncio` são definidos como INTEGER no Sequelize mas armazenam STRING na prática (sigla UF e nome da cidade). Isso precisa ser normalizado.
2. **\*\*Controller monolítico\*\***: `controllers/admin/espacos/index.js` com 7350 linhas é crítico e difícil de manter. Precisa ser modularizado.

3. **\*\*Frontend monolítico\*\***: `Caderno.jsx` com 1504 linhas precisa ser refatorado.
4. **\*\*Dead code\*\***: Após o `return` na linha 544 do controller `listarClassificado`, há ~200 linhas de código morto.
5. **\*\*Google Maps API key hardcoded\*\*** no frontend.
6. **\*\*Duas fontes de seed de UF\*\***: `uf\_full.sql` (com `id\_pais=10`) e `seed\_uf.sql` (com `id\_pais='BRA'`).
7. **\*\*Proxy de imagens\*\***: Dependência do servidor antigo (`minisitio.com.br`) para imagens que não existem localmente. Idealmente todas as imagens devem ser migradas.
8. **\*\*Redis não configurado em produção\*\***: O Railway não tem Redis provisionado, então o cache não está ativo. Os erros "ECONNREFUSED 127.0.0.1:6379" aparecem nos logs.
9. **\*\*Testes\*\***: Não há testes automatizados (backend `"test": "echo Error"`).

### ### 14.2 Sugestões para v3

| Área     | Sugestão                                                         |
|----------|------------------------------------------------------------------|
| Backend  | Modularizar controller de 7350 linhas em serviços e repositórios |
| Backend  | Normalizar FKs: `codUf` → INT, `codCaderno` → INT                |
| Backend  | Remover código morto                                             |
| Backend  | Adicionar testes (Jest + supertest)                              |
| Backend  | Migrar para TypeScript                                           |
| Frontend | Refatorar Caderno.jsx (1504 linhas)                              |
| Frontend | Remover dependência de Google Maps API key hardcoded             |
| Frontend | Migrar CSS de Bootstrap + Tailwind para solução única            |
| Infra    | Adicionar Redis ao Railway                                       |
| Infra    | Migrar imagens do servidor antigo para volume persistente        |
| Infra    | CI/CD com testes automatizados                                   |
| Dados    | Unificar fontes de seed de UF                                    |
| Geral    | Typescript full-stack                                            |

---

> Documentação gerada em Julho/2026 – Minisitio v2.1.29  
> Base para desenvolvimento da v3